Techniques and Methodologies of Self-Adaptive Software and its Architecture: A Survey

Abdur Rehman Riaz
University Institute of Information Technology
PMAS Arid Agriculture University
Rawalpindi, Pakistan
abdurrehman.ar475@gmail.com

Asma Rauf
University Institute of Information Technology
PMAS Arid Agriculture University
Rawalpindi, Pakistan
asmarauf11@gmail.com

Syed Mushhad M. Gilani*
University Institute of Information Technology
PMAS Arid Agriculture University
Rawalpindi, Pakistan
mushhad@uaar.edu.pk

Muhammad Bilal Bashir
Dept. of Computer Science
Iqra University
Islamabad, Pakistan
bilalbezar@gmail.com

2021 4th International Conference on Computing & Information Sciences (ICCIS) | 978-1-6654-9441-0/21/$31.00 ©2021 IEEE | DOI: 10.1109/ICCIS54243.2021.9676190

***Abstract*—Nowadays, due to the continuously changing environment, modern software systems can work in a dynamic environment. But many changes occur at runtime so, to handle the changes self-adaptive mechanism plays an important role. To encounter the requirement of rapid application development adaptive software is used widely. Adaptive software architecture is based on three steps speculate, collaborate, learn. There are a lot of techniques for adaptive software architecture most appropriate technique is selected by comparing them. In this paper, we perform a survey on different research papers related to the self-adaptive system and self-adaptive architecture. After studied the techniques gaps in the literature are identified and compare those techniques. Different common parameters are identified and considered by studying literature. Then these parameters are used to analyze survey techniques which is helpful for researchers and future directions in this area.**

***Index Terms*—Adaptive system, Adaptive Software, Software Architecture, Self-Adaptation, Dynamically adaptive systems**

I. Introduction

Due to the increasing need for software systems, software products are evolving continuously. Different changes occurred in the market due to the advancement in technology. Due to which changes or requirements need to be continuously adapted to encounter the demand of the customer. The aim behind every developed software is to provide a quality product and improve the quality of service [1]. The self-adaptive software can maintain QoS in changing conditions. The software response is adaptive in the working environment of changing requirements, inputs, issues in hardware and external effects on sensors. Therefore, quick response to the rapidly changing requirements of the customer is a difficult part of the success of any software product and the organization. In case of any change, adaptive software can't stop working even it responds to the current condition. They have ability to check their working and being adaptive when expected results are not achieved [2]. Otherwise, it would be very difficult to meet the market needs at the right time or to exist in the competitive market.

Nowadays, due to the continuously changing environment, modern software systems can work in a dynamic environment [3]. Software system needs to be more flexible because system design is directly connected with system's environment and its goal. The design time prediction can sometimes only be removed dynamically when all goals and environment characteristics are visible. The basic structure of adaptive software is shown in Fig 1. In the below figure it is described that adaptive software has three steps which are speculated, collaborate and learn. In the first step discuss project start-up and how adaptive learning is inserted in the software [4]. The second step contains the core part of the project and in the last step apply quality assurance and deploy the project.

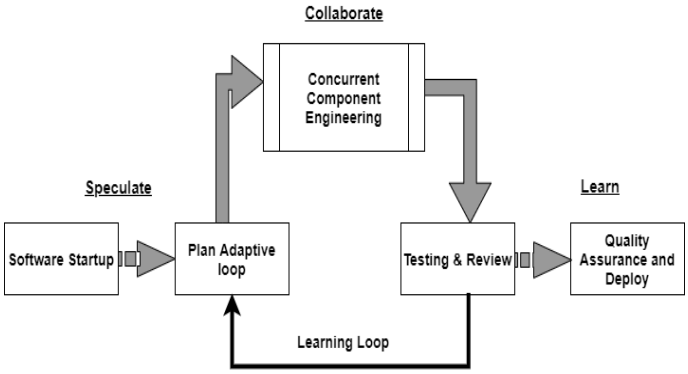

Fig. 1. Implementation of adaptive software.

Current software systems are working in a flexibly changing environment. So, to handle the changes self-adaptive mechanism plays an important role. Devices hardware is changing every day so adaptive applications were used to integrate

*Corresponding Author.

978-1-6654-9441-0/21/$31.00 ©2021 IEEE

with every software [5]. This mechanism helps to manage the changes that occur at runtime that are not easy to predict before the deployment. Probably, self-adaptive software adjusts its behavior after finding the runtime change and accommodate it. An adaptive system is used to make the system secure and able to fight against the risks. Self-adaptive structure from the viewpoint of architecture is made up of two subsystems that are managed and managing systems [6]. Manage system handles the main functions of the system and the managing system handles the managed system by adapting it. To ensure self-adaptability is not a minor task. There are several challenges that software architects need to resolve in adaptive system. Flexible system are needed to used in changing environment [7].

In this research we perform a survey on adaptive software architecture techniques. We select some techniques from previous literature and analyze them by performing a comprehensive critical review on each technique. Then select 6 parameters Risk driven, Change tolerant, Time boxed, Link between Architectural design and decision making, Scalability and scenario. These parameters are elaborated as they present in the technique or not. Compare the selected techniques on the basis of parameters then perform an analysis which is helpful for future research. We distribute the paper as section II describes significant survey techniques which we use in this paper. In section III assessment criteria is provided. Analysis of the assessment factors is given in section IV. Section V presents the conclusion.

II. Surveyed Techniques

A critical review of all the surveyed techniques and methods is described in this section. These techniques and methods describe a proposed solution for Adaptive software architecture.

A. M. A. Pereira da Silva, V. Cesario Times, A. M. Costa da Araujo, and P. Caetano da Silva (2021)

An adaptive Healthcare Software architecture (AHSA) was proposed that offers runtime adaption. The architecture is composed of three components that work together to process the data and to allocate the right component for processing the data. A tool has been developed to produces the adaptive healthcare software architecture. The algorithm that use in the proposed architecture help to represent the production of its components. Two real-world scenarios were used to examine the adaptability of HIS by doing a comparison with the other five tools. The result shows that HIS based on adaptive software architecture can adapt at runtime to process changes in software requirements and provide more adaptability than others. There is a need to evaluate the AHSA's adaptability in case of a system fault or if any change occurred in an environment [8].

B. Bodnarchuk, Ihor, et al (2018)

Due to the increasing use of software, the demand for the development of mobile applications has also increased. For this reason, from the last few years, companies have shifted their trend of development towards flexible approaches such as scrum, extreme programming, Kanban for the development of large projects. In this paper, the researcher explained that the wide use of the flexible technique for the development of the software shows concerns towards the quality of the software. They explained the combination of flexible techniques and classical architecture design.

In this paper the researchers provided an approach in which they use goal function which is used to control and manage the quality of the software architecture at each iteration of the software development step [9]. In this paper to handle the non-satisfactory result, the researcher provided the approach in which they use optimization methods and tools to redesign the architecture to automate assessment operation and to select the appropriate software architecture from the set of architectures. In this study, the researcher has considered only a few parameters for architecture quality criteria. There may be a possibility of getting more good results by changing the values of the quality criteria parameters [10].

C. A. Bershadsky, A. Bozhday, Y. I. Evseeva, and A. Gudkov (2018)

The concept of independent computing is useful to build large software and hardware in a real-time environment. The traditional circumstances of automatic control theory are adaptive behavior. The study handles the issue of making mathematical apparatus to implement the properties of self -adaptive software system. One of the Self-adaptability is the ability to change the behavior and structure of software under the control of external components.

The methodological limitation is one of the prominent problems in the existing approaches because in the existing approaches the overall parts of advanced achievement in IT have not been used. The newly proposed model helped to build various types of self-adaptive software for various purposes. The self-optimization level of the model help to identify error directly during the functioning of the process and the self-adaption level of the model help to make new response during the life cycle due to which flexibility of program application has also increased [11].

D. N. Abbas, J. Andersson, and D. Weyns (2020)

The need for the self-adaptive system has increased over the last few decades. Self-adaption helps to handle the changes that occur at run which are difficult to predict before the deployment. A lot of research has been done on the self-adaptability of software but there is a need to provide detailed knowledge of process support for self-adaptive system development with reuse. In this paper domain, the engineering-based methodology was developed to provide a guideline to develop families of SASS with structured reuse. In this methodology, different roles were defined to frequently assist the development of the system and to improve the quality and increase the reuse. The result of the case study for the evaluation of design methodology for self-adaptive systems shows the positive effect on the quality and reuse of the system

as compared to other engineering practices. There is need to identify the problems and provide solution for those problems that occur during the case study [12].

E. F. A. Moghaddam, R. Deckers, G. Procaccianti, P. Grosso, and P. Lago (2017)

Modern software systems can work in an environment that changes continuously. The focus of previously developed self-adaptive software systems was either on runtime adaption or designing the self-adaptive system. Several modeling frameworks have been introduced to realize self-adaptive software systems. Due to which the link between the system and architecture concept has not been given properly. The main reason to provide a link between these two is to ensure that system has sent the right design for self-adaptability. For this purpose, the researcher proposed a domain model to assist both system engineering and architecture design for the self-adaptive system. The suitable concepts at runtime and design time were used to support the self-adaptive software design. In this paper the researcher need to evaluate the self-adaptive system based on the design domain model quantitatively [13].

F. T. Brand and H. Giese (2018)

The contribution of this paper is a set of elaborated requirements for a modeling language to create and maintain evolvable runtime models that support continuous adaptive monitoring of a running system. A prospective approach describes different illustrative scenarios which derive and generalize the requirements. Additionally, investigate two state-of-the-art approaches regarding their support for the scenarios and requirements to identify beneficial and missing features. First, the paper describes illustrative scenarios which can occur with systems employing runtime models.

With the scenarios, it is considered, for example, experimentation in software product development and realizing self-adaptive systems utilizing machine learning. Two state-of-the-art modeling languages approach namely the classical model-driven engineering approach and an existing, reusable, and rather a common modeling language for dynamic modeling. It is found that the flexibility which the CompArch modeling language provides by allowing the definition of classifiers in the runtime model is particularly helpful to support the stated scenarios. There is a need to work towards the prospective runtime model modeling language, for example, by evaluating different implementation options including the use of the dynamic object model pattern [14].

G. N. Ali and J.E. Hong (2017)

Due to the continuous evolvement of the software product to meet the market needs the emergence of new requirements and the quick change in the context are a common part of software products. Various tools and techniques have been proposed to handle the problem of adaptive architecture in software product line but no systematic process is available that provide step by step by step guidelines and task for the development of dynamic software architecture. To systematically map the activities a process was designed to develop an adaptive architecture systematically. The variability-driven approach is used to build a dynamic adaptive architecture. In the research paper variability monitoring agent is introduced to examine the change and decide whether reconfiguration of architecture is required or not. The researchers have not explained the problem that occurs during the implementation of adaptive architecture and they have not defined any solution to support uncertain features [15].

H. A. Mehmood and Y. Gulzar (2020)

This paper using an approach that combining aspect-oriented software development and architectural modal for modeling dynamically adaptive modeling systems. Two modeling techniques are used in two different parts, First part developing a declarative model that focuses on the working and implementation. For studying the adaptive behavior reconfiguration model is used in the second part. Both parts are working together to achieve the runtime adaptation. The first part that is the declarative model is working in two steps. The Architecture Analysis and Design Language (AADL)is used in the steps of the declarative modal. AADL is a well-known modeling language use for modeling the whole system and also define its connections.

The first step of the declarative model describes the development of the deployment-agnostic functional model in which components of the system that require execution are not required at that time. The next step model is talking about deployment which artifacts are needed for the development. This model is used as a specimen for the systems. This paper using a case study for validating this model. The paper also describes some issues of adaptive application development in IoT. They are focus on integrating unanticipated reconfiguration which requires manual handling they try to convert manual handling into a dynamic model [16].

III. Assessment Criteria

This section presents the assessment criteria for our above analysis techniques of adaptive software architecture. We discuss them in detail below by finding some parameters in our technique.

A. Risk Handling

New challenges have come in adaptive software development and for every iteration, there is a need to solve and process high-risk activities rapidly. Table I describe the assessment criteria for the risk handling.

TABLE I
Assessment criteria for Risk Handling

Criteria	Description
Yes	The technique can fix high risks activities quickly
No	The technique cannot fix high risks activities quickly

The iterations of the adaptive system are identified and analyzed so that we implement the risks evaluation techniques

on the software. New changes in the system come up with new risks.

B. Change Tolerant

Adaptive softwares are changing continuously due to the external environment so by little planning and self-learning of the system we made it accept the change. Change is not a problem in an adaptive system because the system already has some planning to handle the change. Iteration in the system is not considered a problem it is considered a new challenge. Assessment criteria for the Change Tolerant are described in Table II.

TABLE II
ASSESSMENT CRITERIA FOR CHANGE TOLERANT

Criteria	Description
Yes	The system is able to bear the changes
No	The system is not able to tolerate the changes

C. Time Boxed

Time boxed means not that to fixing or delivering something on time but it is response to the change on time. When a system changes very rapidly and its change rate is high it can settle on time without affecting the quality of the system. Table III describes assessment criteria for the Time Boxed.

TABLE III
ASSESSMENT CRITERIA FOR TIME BOXED

Criteria	Description
Yes	Time is set to deliver the project minimally
No	No time is set to deliver the product

D. Link between architectural design and decision making

Current software systems are enough smart to work in a changing environment and respond according to conditions. By adding decision-making in software is able the software to work self-adaptability and handle the situation. It means that levels of both architecture and decision-making are considered while making the adaptive system. Table IV evaluated this parameter.

TABLE IV
ASSESSMENT CRITERIA FOR LINK BETWEEN ARCHITECTURAL DESIGN AND DECISION MAKING

Criteria	Description
Yes	The system has an appropriate link between the architectural design and decision making
No	Proper link is not provided between the architectural design and decision making

E. Scalability

To cater to the market needs or to accomplish the demand of the customer continuous adaption is required. So for this reason there is a need to see that whether the architecture of the self-adaptive system performs well if it is expanded. How scalability is assessed describes in Table V.

TABLE V
ASSESSMENT CRITERIA FOR SCALABILITY

Criteria	Description
Yes	The system performs well if it expanded
No	The system does not perform well if it expanded

F. Scenarios

To check the technique in the real world how it works and perform scenarios are used. A practical example is taken and the technique is performed and calculate the results. These scenarios give very good results if these are performed without biasness and results are almost the same in different situations. For checking the adaptive behavior of techniques scenario building is working well. Table VI narrate assessment criteria for scenario parameter.

TABLE VI
ASSESSMENT CRITERIA FOR SCENARIOS

Criteria	Description
Yes	If the scenario is performed on the technique of Adaptive system
No	The system does not perform scenario on the technique of Adaptive system

IV. ANALYSIS OF SURVEYED TECHNIQUES

Different parameters are considered by studying different techniques from different papers. The parameters that are considered to analyze the techniques are Risk driven, Change tolerant, Time boxed, Link between Architectural design and decision making, Scalability and scenario. In [8] the defined technique can handle the risk and tolerates the change. In this technique, a scenario is used. The technique that is used in the [10] handles the risk quickly. The relevant concepts are defined to support the architecture design at runtime and no time frame is provided for the delivery of the project. In this paper, the link between architectural design and decision-making has also been discussed. No scenario is used in this technique. The method that is used by authors of [11] helps to quickly accommodate the need of the market and handle the challenges according to the situation and no time boxed is a lot at which the system should be delivered. The technique used in this research accommodates the changes and it also performs well when this technique is used on large scale. The overview of all techniques analysis is shown in Table VII. In [12] the techniques do not handle the risk but can tolerate the change, the defined technique provides a link between the architectural design and design decision. In this technique, a scenario is used.

TABLE VII
ANALYSIS TABLE OF SURVEYED TECHNIQUES OF ADAPTIVE SOFTWARE ARCHITECTURE

Technique	Change tolerant	Time boxed	Risk driven	Scalability	Link between Architectural design and decision making	Scenario
pereira et al. [8]	Yes	Yes	No	No	Yes	Yes
bodnarchuk et al. [10]	Yes	No	Yes	No	No	No
bershadsky et al. [11]	No	Yes	No	No	Yes	No
abbas et al. [12]	Yes	No	Yes	Yes	No	No
moghaddam et al. [13]	Yes	No	Yes	Yes	Yes	No
brand et al. [14]	No	Yes	Yes	Yes	No	Yes
ali et al. [15]	Yes	No	Yes	Yes	No	No
mehmood et al. [16]	Yes	No	Yes	Yes	No	No

The studies in [13] describe a technique that is used that can accommodate the changes and handles the risk. This paper has not explained that whether this technique is suitable when it is expanded or use on large scale. This technique does not provide a link between architectural design and decision-making. The authors of [14] used the technique that is poor to tolerate the changes and handle the risk. A scenario is used in this paper. It can not link decision-making and architectural design. In the studies [15] the technique can handle the risk but does not tolerate the changes, this technique provides a link between architectural design and decision making. It is fine in the feature of time boxed and also uses a scenario. In [16] the technique that is used tolerates the change and performs well if it is expanded. It is working well in risk handling and scalability but doing not accurate in time-boxed and linking between architecture and decision making.

V. CONCLUSION AND FUTURE WORK

Different changes occurred in the industry due to the advancement in technology. Due to which different problems occur at runtime in a continuously changing environment. For this reason, the self-adaptive system plays a vital role to handle the runtime changes. The purpose of this study is to view the solutions that are presented in a different paper to handle the run time dynamics. So, for this reason, different papers are studied and the gaps in the research paper are also identified. Then different parameters are considered based on the studied literature and based on selected parameters. Techniques that are used in the studied literature are evaluated. After this, analysis performs to evaluate the techniques by using some common factors. Almost all the techniques handle the risk and are change tolerant. In this paper limitations are also identified in the research papers that are considered for literature. These limitations are discussed in the survey of every technique.

ACKNOWLEDGMENT

We gratefully acknowledge faculty of "Arid Agriculture University UIIT department" for their support and guidance in every stage of this research paper.

REFERENCES

[1] Cámara, Javier, Henry Muccini, and Karthik Vaidhyanathan. "Quantitative Verification-Aided Machine Learning: A Tandem Approach for Architecting Self-Adaptive IoT Systems." IEEE International Conference on Software Architecture (ICSA). IEEE, 2020.

[2] San Martín, Daniel, and Valter Camargo. "A Domain-Specific Language to Specify Planned Architectures of Adaptive Systems." 15th Brazilian Symposium on Software Components, Architectures, and Reuse. 2021.

[3] Cha, Jung-Eun, Jeong-Si Kim, and Young-Joon Jeong. "Architecture Based Approaches Supporting Flexible Design of Self-Adaptive Software." International Conference on Computational Science and Computational Intelligence (CSCI). IEEE, 2016.

[4] Huynh, Ngoc-Tho. "Towards Automatically Generating State Transfer Model Integrated in Adaptive Software Development Process." International Journal of Applied Engineering Research 15.2 (2020): 189-196.

[5] Motger de la Encarnación, Joaquim, Javier Franch Gutiérrez, and Jordi Marco Gómez. "Integrating adaptive mechanisms into mobile applications exploiting user feedback." Research Challenges in Information Science, 15th International Conference, Springer, 2021.

[6] D. S. Martin, B. Siqueira, V. V. de Camargo, and F. Ferrari, "Characterizing Architectural Drifts of Adaptive Systems," IEEE 27th International Conference on Software Analysis, Evolution and Reengineering (SANER). London, ON, Canada: IEEE, Feb. 2020, pp. 389399.

[7] Frangoudis, Pantelis A., Matthias Reisinger, and Schahram Dustdar. "Recursive design for data-driven, self-adaptive IoT services." IEEE International Conference on Service-Oriented System Engineering (SOSE). IEEE, 2021.

[8] M. A. Pereira da Silva, V. Cesario Times, A. M. Costa da Araujo, and P. Caetano da Silva, "To- wards an Adaptive Software Architecture for Archetype-Based Healthcare Applications," IEEE Software, 2021, pp. 18.

[9] Bashir, Muhammad Bilal, and Aamer Nadeem. "Object oriented mutation testing: A survey," International Conference on Emerging Technologies. IEEE, 2012.

[10] Bodnarchuk, Ihor, et al. "Adaptive Method for Assessment and Selection of Software Architecture in Flexible Techniques of Design." IEEE 13th International Scientific and Technical Conference on Computer Sciences and Information Technologies (CSIT). Vol. 1. IEEE, 2018.

[11] A. Bershadsky, A. Bozhday, Y. I. Evseeva, and A. Gudkov, "The Mathematical Model of Reection for Self-Adaptive Software," 9th International Conference on Information,Intelligence, Systems and Applications (IISA). Zakynthos, Greece: IEEE, Jul. 2018, pp. 15.

[12] N. Abbas, J. Andersson, and D. Weyns, "ASPLe: A methodology to develop self-adaptive soft- ware systems with systematic reuse," Journal of Systems and Software, vol. 167, p. 110626, Sep. 2020.

[13] F. A. Moghaddam, R. Deckers, G. Procaccianti, P. Grosso, and P. Lago, "A domain model for self-adaptive software systems," in Proceedings of the 11th European Conference on Software Architecture: Companion Proceedings. Canterbury United Kingdom: ACM, Sep. 2017

[14] T. Brand and H. Giese, "Towards software architecture runtime models for continuous adaptive monitoring." in MODELS Workshops, 2018, pp. 7277.

[15] N. Ali and J.E. Hong, "Creating adaptive software architecture dynamically for recurring new requirements," International Conference on Open Source Systems and Technologies (ICOSST). Lahore: IEEE, Dec. 2017, pp. 6772.

[16] A. Mehmood and Y. Gulzar, "An Architecture-Based Approach for Modeling Dynamically Adaptive IoT Systems," in 2020 International Conference on Computing and Information Technology (ICCIT-1441). Tabuk, Saudi Arabia: IEEE, Sep. 2020, pp. 16.